TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA



Gestión de Desarrollos de Software

Unidad 5 - Práctica

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13

Repositorio GitHub: /Lucenear/UTN-TUPaD-TPs/Gestion_de_Desarrollos_de_Software

1- Organizacion de artefactos y principios de documentacion

Estructura de carpetas:

- Archivo de variables de entorno (.env.example): config/
- Imagenes de capturas de pantalla para el manual: assets/
- Codigo fuente de los controladores de autenticacion: src/
- Pruebas unitarias de integracion: tests/
- Especificacion tecnica del sistema en Markdown: docs/

Calidad de la documentacion. Las 4 C de la buena documentacion son:

- Clara: Lenguaje sencillo, directo y sin ambiguedades.
- Concisa: Ir al grano, sin informacion redundante.
- Correcta: Informacion precisa, actualizada y sin errores tecnicos.
- Completa: Contener todo lo necesario para que el usuario logre su objetivo.

Aplicacion en el README.md: Se debe escribir con encabezados claros, pasos concisos para la instalacion y uso, asegurar que los comandos listados sean correctos y actuales, y cubrir desde la clonacion del repositorio hasta la ejecucion de las pruebas, dejando todo completo en un solo archivo.

2- Comparativa: Documentacion tradicional vs. Docs-as-Code

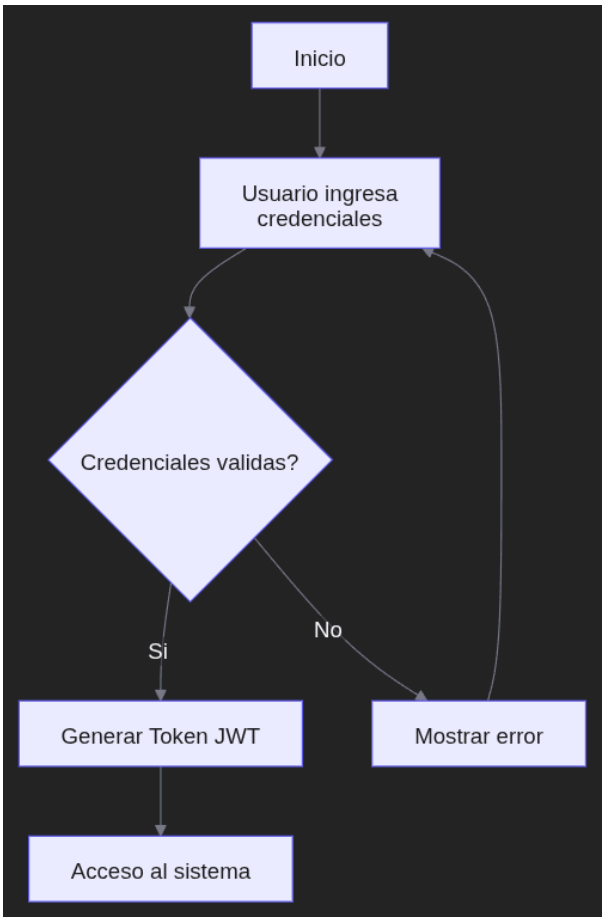
Aspecto	Documentacion Tradicional	Enfoque Docs-as-Code
¿Donde se escribe/almacena?	Archivos Word/PDF o wikis externas separadas del codigo	En el mismo repositorio que el codigo, usualmente en la carpeta docs/ en formato Markdown
Control de versiones	Nulo o manual, dificil de rastrear	Registro mediante Git (commits, historial, autor, ramas)
Revision de cambios	Comentarios manuales o envios por correo electronico	Revision colaborativa mediante Pull Requests con comentarios en linea
Publicacion	Manual, requiere que alguien suba el archivo a la wiki o servidor	Automatica mediante pipeline CI/CD tras cada merge a la rama principal
Formato del archivo	Binario / Procesador de texto (DOCX, PDF)	Texto plano (Markdown, AsciiDoc)

3- Diagramacion como Codigo (Mermaid.js) y Mantenibilidad

Analisis del problema:

El documentation drift o deriva de la documentacion es el fenomeno donde la documentacion queda desactualizada respecto al codigo fuente a medida que el software evoluciona. Los diagramas en formatos de imagen tradicionales (.png, .jpg) favorecen este problema porque son archivos binarios. Git no puede mostrar que lineas cambiaron en una imagen, lo que dificulta la revision de cambios y hace que los desarrolladores olviden o eviten actualizar los diagramas al modificar el codigo.

Construccion en Mermaid:



Explicacion de texto plano:

Guardar el diagrama como texto plano dentro del repositorio facilita la colaboracion mediante Pull Requests porque Git puede rastrear los cambios linea por linea. Los revisores pueden ver exactamente que nodo o conexion se modifico en el diff del Pull Request, permitiendo discusiones tecnicas precisas sobre la arquitectura y el diseño antes de aprobar el cambio, algo imposible con imagenes.

4- Manuales, Instructivos y Procedimientos Operativos Estandar (POE)

Tipologia de manuales:

- Un nuevo desarrollador se suma al equipo y necesita configurar su entorno local y la base de datos: Manual de Instalacion (o Guia de Configuracion).
- Un administrador de sistemas necesita consultar la arquitectura de componentes, esquemas de BD y endpoints de la API: Manual Tecnico.
- Un cliente final necesita aprender a navegar por el sistema y realizar las tareas cotidianas en lenguaje simple: Manual de Usuario.

5- Redaccion Generativa y Automatizacion con IA

Integracion de la IA en el pipeline:

La Inteligencia Artificial Generativa se integra en el pipeline de Docs-as-Code mediante hooks o acciones automatizadas. Por ejemplo, al detectar cambios en el codigo, un script puede enviar el codigo modificado a un modelo de IA para que genere o actualice automaticamente el borrador del manual en Markdown. Para diagramas, la IA puede traducir descripciones textuales o cambios en el codigo directamente a sintaxis Mermaid.js, actualizando la documentacion visual sin intervencion manual inicial.

Beneficio clave y limitacion:

- Beneficio clave: Acelera drasticamente la creacion y actualizacion de borradores, manteniendo la documentacion mas sincronizada con el codigo y reduciendo la carga cognitiva de los desarrolladores.
- Limitacion o riesgo: El riesgo de alucinaciones. Si la IA no tiene el contexto adecuado del sistema, puede inventar pasos, endpoints o comportamientos que no existen. Por esto es vital el enfoque human-in-the-loop, donde un experto humano siempre debe revisar y validar el contenido generado antes de su fusion y publicacion.